CNENGR202 Instrumentation and Control

Part 3 L5 – Interrupting

Prof. Zhang Liwei 张立伟 教授

Email: lwzhang@bjtu.edu.cn
Working Room: SX702

School of Electronics and Information Engineering
Beijing Jiaotong University

Recap

- 16 MHz too fast for humans!
- Delaying tactics
  - Implement delays by means of loops are ok, but accuracy, flexibility?
  - Timers are flexible and accurate …. delays()

Brief

- Delay() brings CPU to a grinding halt!
- Constant polling to see if timer has overflowed
- Wasteful of computational power and time

- Do something interesting, gets interrupted when there is an event

Interrupts

- Interrupts are actions that pull us away from our normal activities
  - Phone Ringing
  - Door Bell
  - Smoke Alarm
  - Microwave Oven
- Microcontrollers have Interrupts that signal when they need attention
  - Timer Interrupt
  - Serial Interrupt
  - External Interrupts
  - etc.

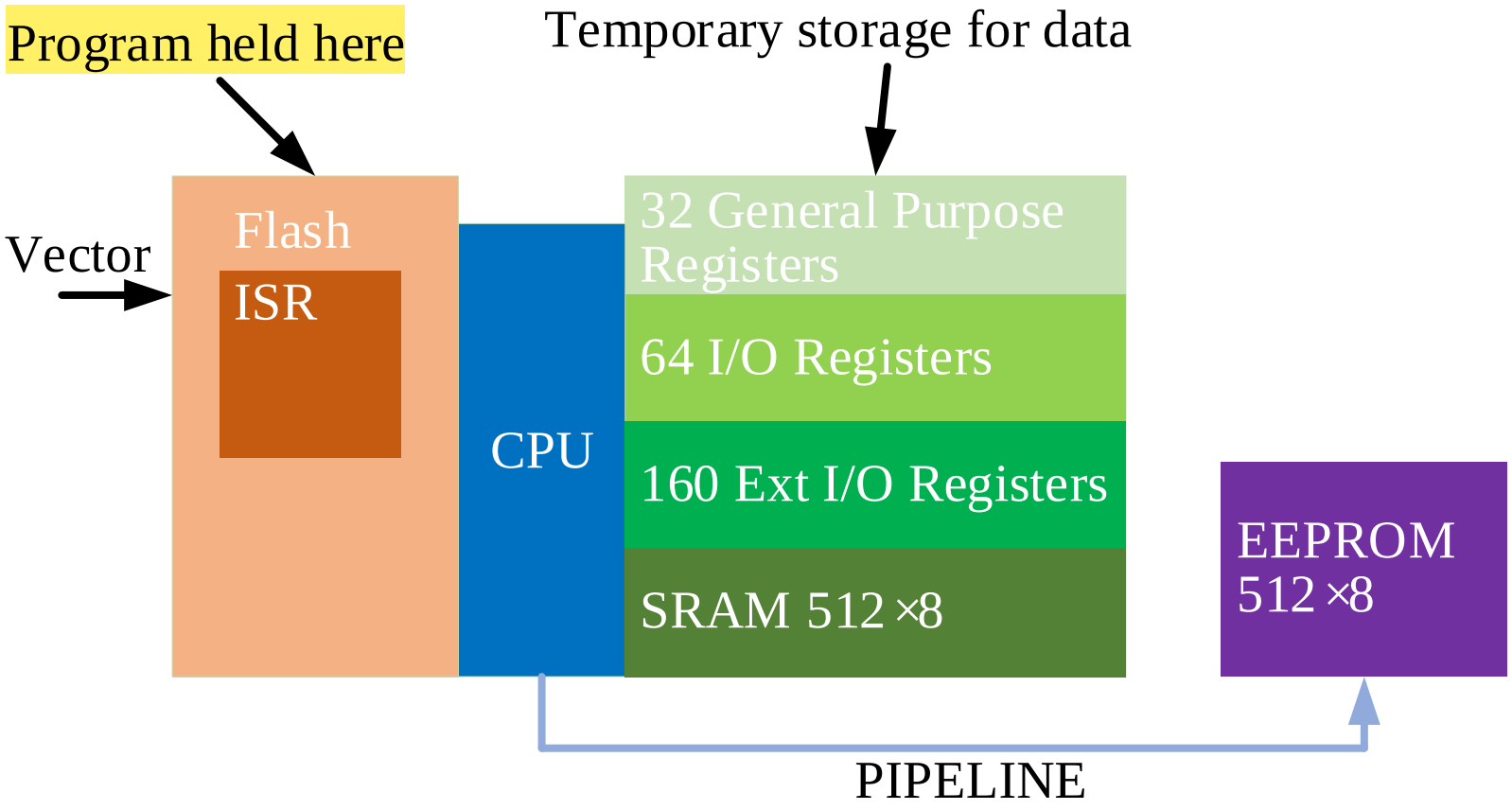
Interrupts

- Why the interrupts are important?
  - Coordinate the input and output activities
  - Prevent the CPU from being tied up
  - Permit to exit from errors in an effective manner
  - Remind the CPU to perform routine tasks

What happens on an interrupt?

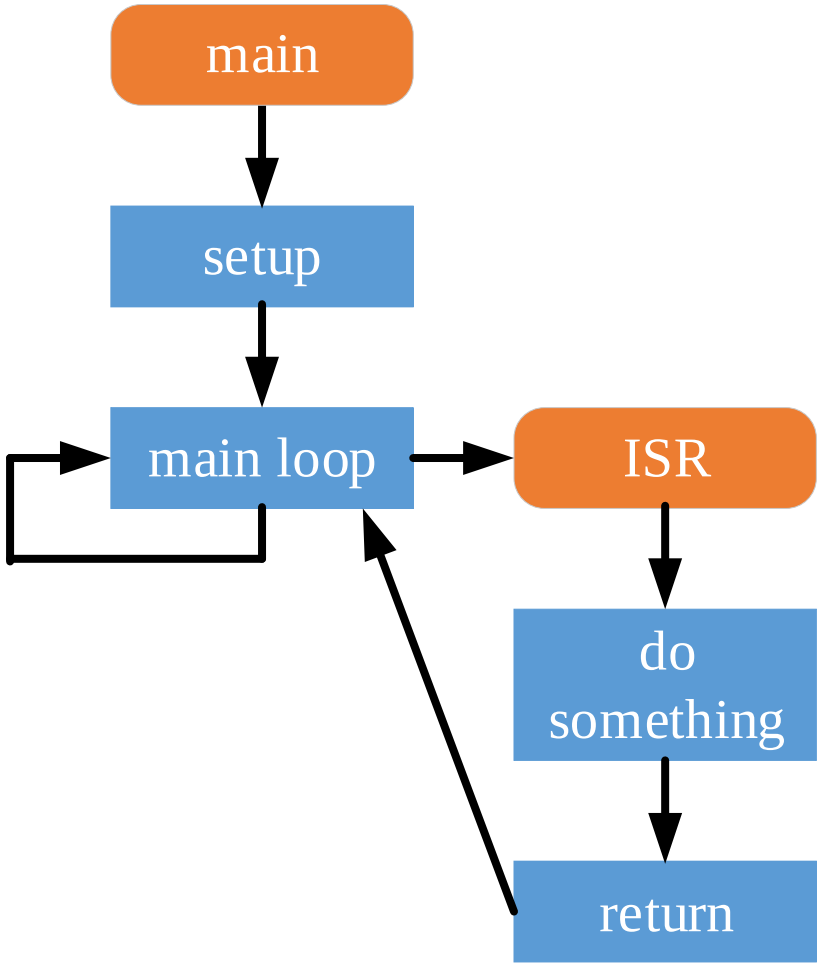
- The CPU attempts to jump to a predefined program location defined for handling the particular interrupt
- This location is called a vector
  - Like a label
  - Predefined rather than free to chose
  - Refers to a specific address in the program memory
- This means the processor can quickly jump execution when required

ATMega Memory - map



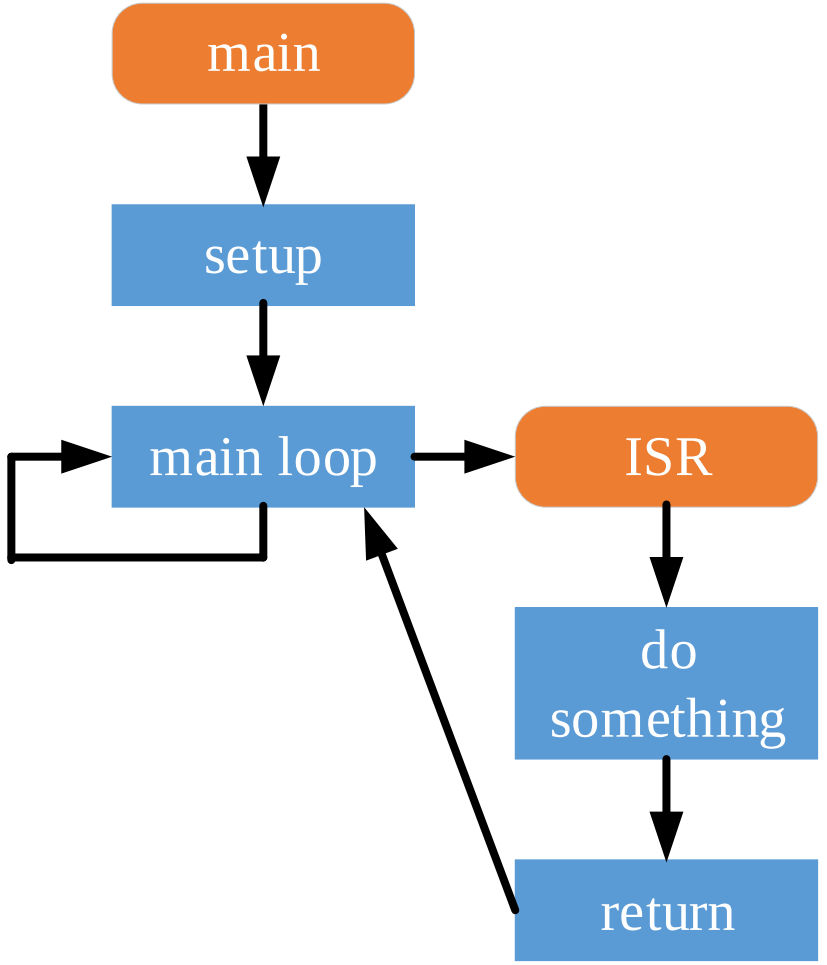
A typical program - I

- ISR
  - Interrupt Service Routines (ISR) are functions that are executed in response to a hardware interrupt



A typical program - II

- ISR are almost identical to normal functions with a few executions
  - They CANNOT receive any parameters
  - They CANNOT return any values
  - WHY?
- We do not know when they are going to occur!!



Declare ISR using a vector

- ISR command tells the compiler where to put the code that follows it (using a vector)
- Lookup name of vector
- When the routine is finished, return to the point in the program interrupted

```
ISR (VECTOR) {
        do something
}
```

Advantages

- No polling or delay (CPU can do other things)
- Immediate handling of interrupts
- Can block other code, then return

Limitations of interrupt processing

- The space for ISR code is limited.
- This is because ISRs are strictly designed to perform short servicing tasks, clear flags etc.
- It is not good programming style to include extensive processing in an ISR and major tasks ought to be referred back into the main program.

Interrupt sources

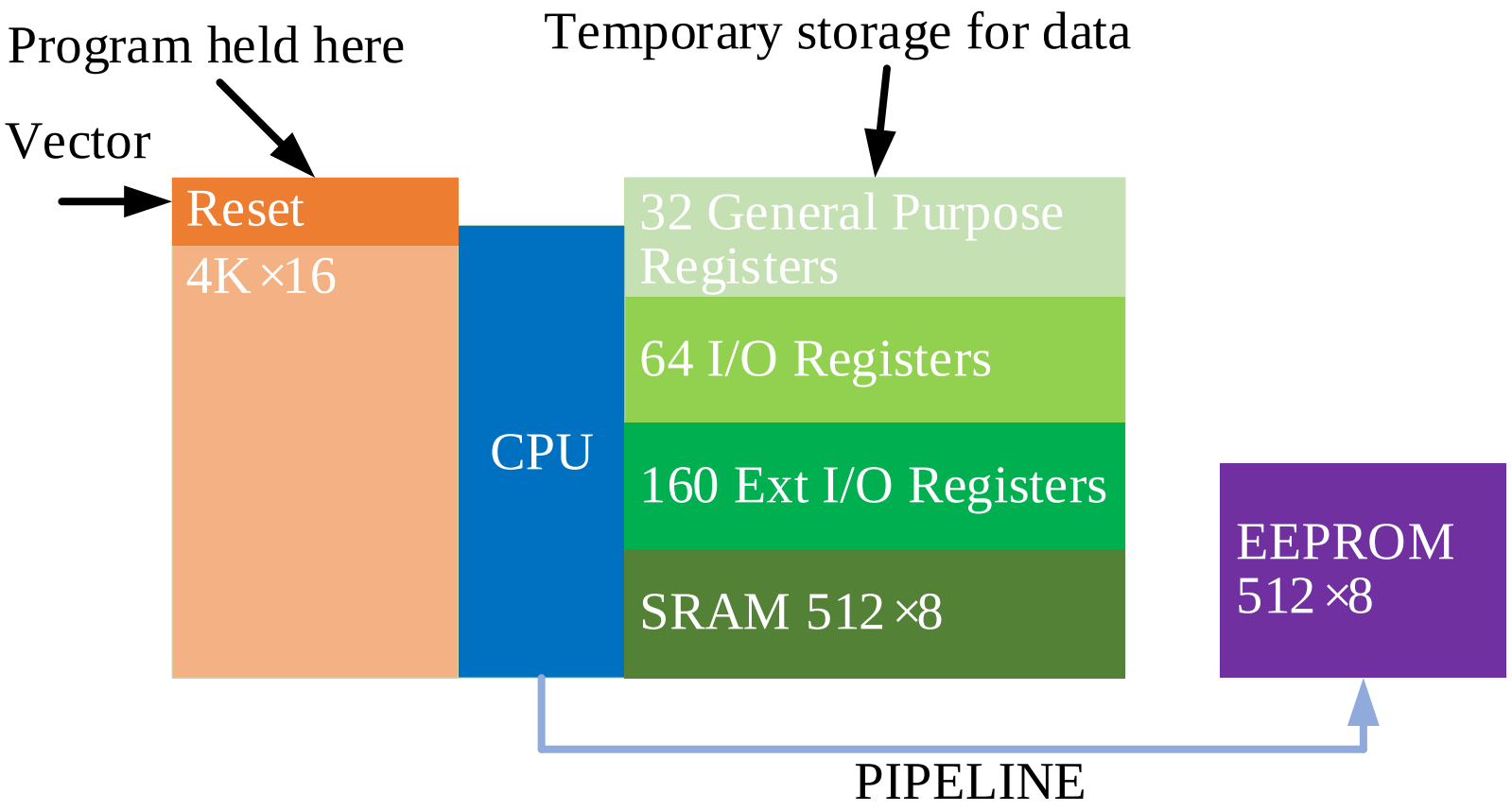
- Internal
  - Reset
  - Timers etc
- No dedicated function

- External
  - Push a button
  - Pin signal change
- Dedicated function
  - attachinterrupt()
  - detachinterrupt()

Reset

- Reset is an interrupt
- Set the initial conditions for the microcontroller
  - CPU Registers
  - Control Registers
  - Modules (timer, adc)
- The main reset mechanisms
  - The power - on reset
  - The manual reset without power - down allows the computer to get out of most error conditions if hardware doesn't fail
- Reset cannot be ignored

ATMega Memory - map



Vector No.	Program address[2]	Source	Interrupt definition
1	0x0000[1]	RESET	External Pin, Power on Reset, Brown out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time out Interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2 OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1 COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 Compare Match B
14	0x001A	TIMER1 OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 Compare Match B
17	0x0020	TIMER0 OVF	Timer/Counter0 Overflow
18	0x0022	SPI, SPC	SPI Serial Transfer Complete
19	0x0024	USART, RX	USART Rx Complete
20	0x0026	USART, UDRE	USART, Data Register Empty
21	0x0028	USART, TX	USART, Tx Complete
22	0x002A	ADC	ADC Conversion Complete
23	0x002C	EE READY	EEPROM Ready
24	0x002E	ANALOPG COMP	Analog Comparator
25	0x0030	TWI	2-wire Serial Interface
26	0x0032	SPM READY	Store Program Memory Ready

Priority

Interrupt sources

- Sources
  - Timers
  - Pins
  - ADC
  - Various communication ports
- Priority
  - If we have multiple interrupts occurring at the same time, the one with the **highest vector** gets serviced first.

Vector definitions

ISR (VECTOR) {
do something
}

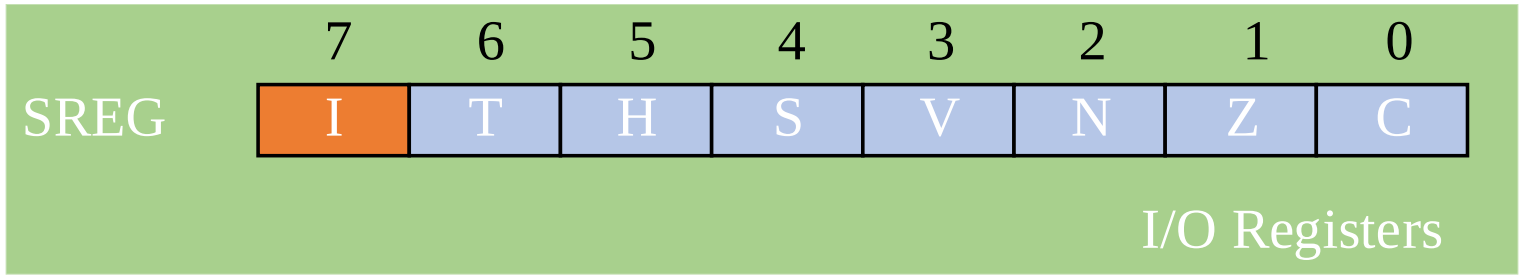
Vector No.	Program address	Source	Interrupt definition
1	0x0000	RESET	External Pin, Power on Reset, Brown out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time out Interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2 OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1 COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 Compare Match B
14	0x001A	TIMER1 OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 Compare Match B

Masking interrupts

- Interrupt maskability
  - Maskable interrupts are interrupts that can be ignored
  - A maskable interrupt must be enabled before it can interrupt
  - A nonmaskable interrupt is an interrupts that can't be ignored by the CPU.
- ALL interrupts are masked by default
  - except reset

Enabling Global Interrupt

- Interrupts are by default disabled



- I: Global Interrupt Enable (enabled when 1)
- To set “Global Interrupts Enable” use the command

  SEI (set global interrupt flag)

- To set “Global Interrupts Unenable” use the command

  CLI (clear global interrupt flag

Masking Interrupts

- Enabling an interrupt takes two steps
  1. Enable interrupts globally
  2. THEN enable the specific interrupt
     - different method for each interrupt

Declare ISR using a vector

ISR (VECTOR) {

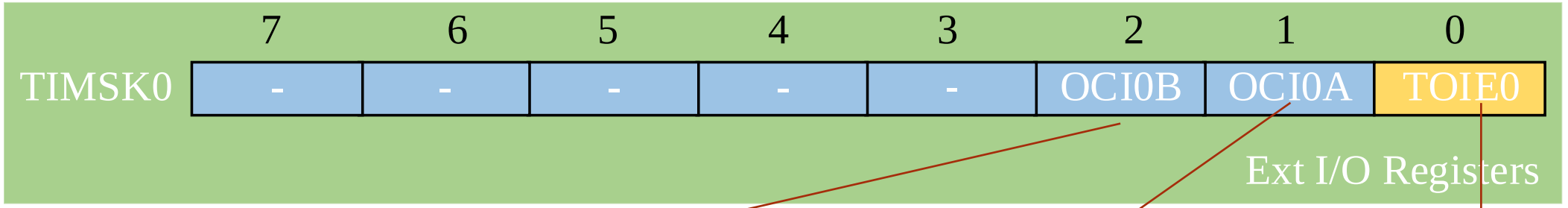
do something

}

Vector No.	Program address[2]	Source	Interrupt definition
1	0x0000[1]	RESET	External Pin, Power on Reset, Brown out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time out Interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2 OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1 COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 Compare Match B
14	0x001A	TIMER1 OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 Compare Match B

Enable timer interrupts - I

- Don’t forget SEI or interrupts() (to enable global interrupts)
- Enable the timer interrupts (Timer Interrupt Mask Register)



Timer/Counter Output Compare Match B Interrupt Enable

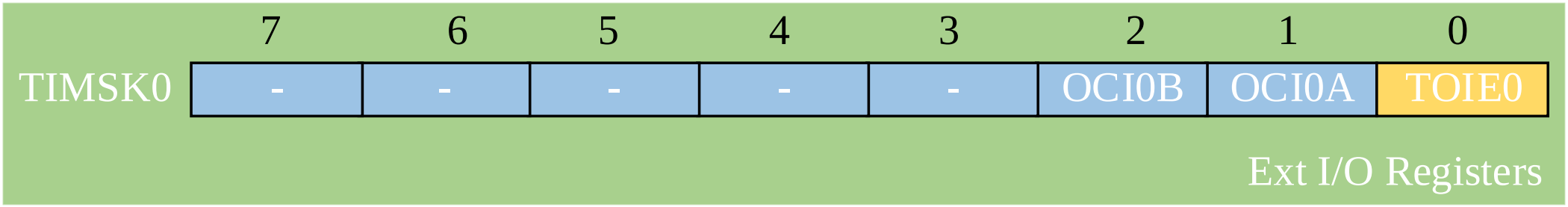
Timer/Counter0 Output Compare Match A Interrupt Enable

Timer/Counter0 Overflow Interrupt Enable

- Set this bit to allow the timer interrupts to happen

Enable timer interrupts - II

- Enable the timer interrupts



- To do this, many ways eg
  - TIMSK0 |= (1 << TOIE0);
  - TIMSK0 = 1;
  - TIMSK0 = 0b00000001;

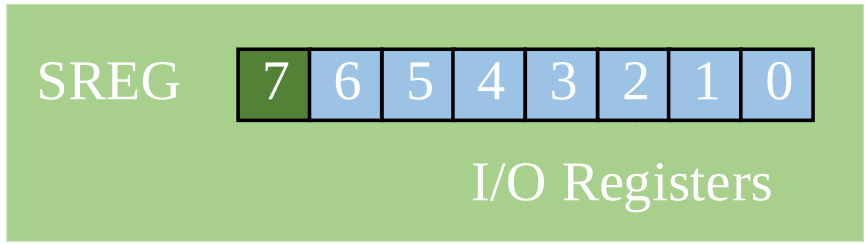
Vector definitions

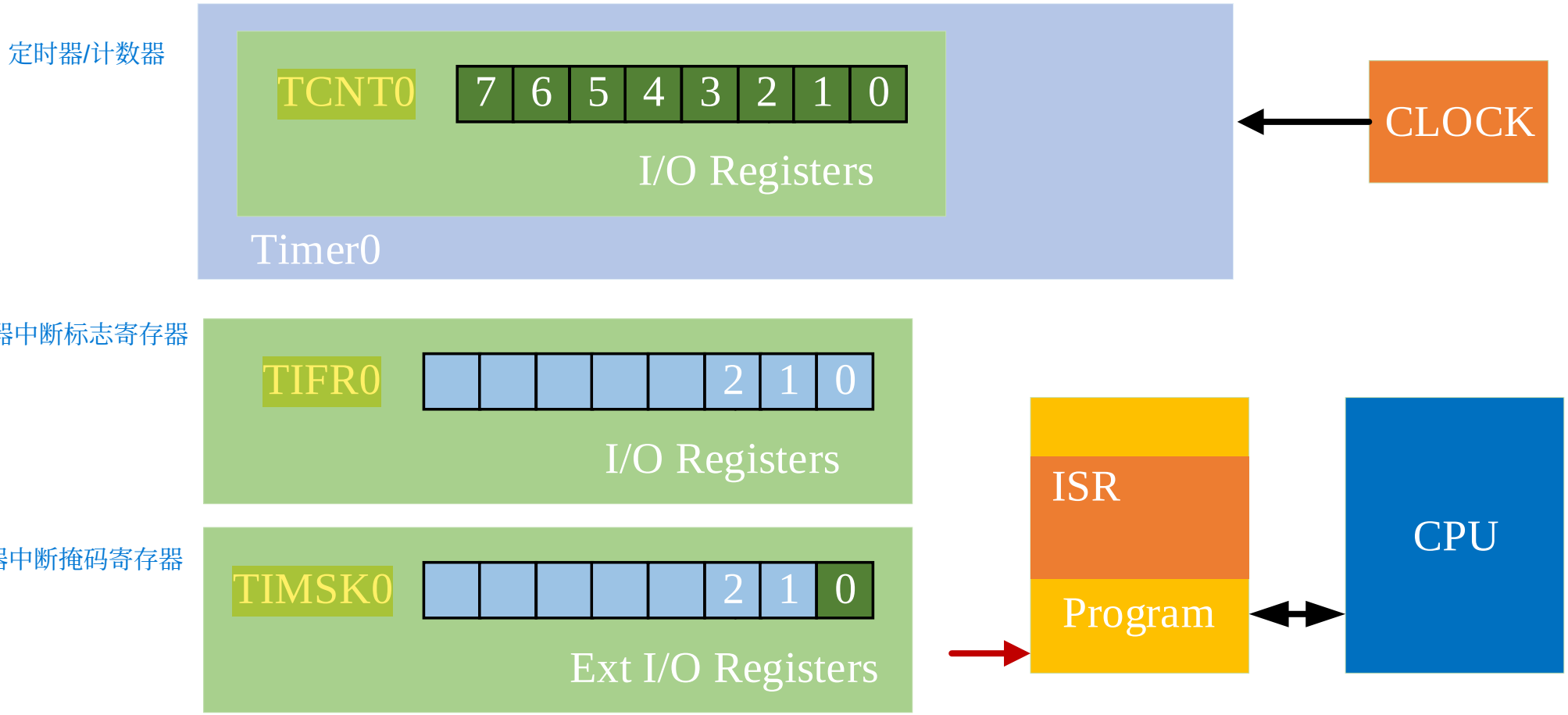
ISR (VECTOR) {
do something
}

Vector No.	Program address[2]	Source	Interrupt definition
1	0x0000[1]	RESET	External Pin, Power on Reset, Brown out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time out Interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2 OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1 COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 Compare Match B
14	0x001A	TIMER1 OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 Compare Match B

Timer ISR - I

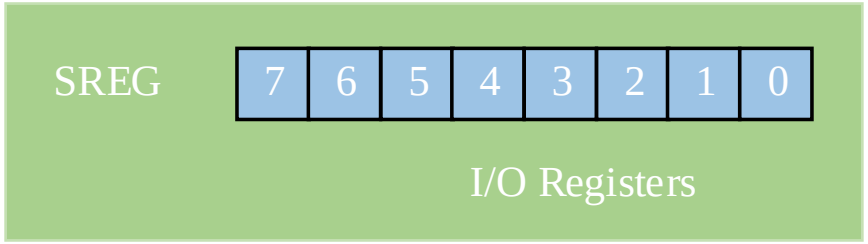
- Reach maximum value

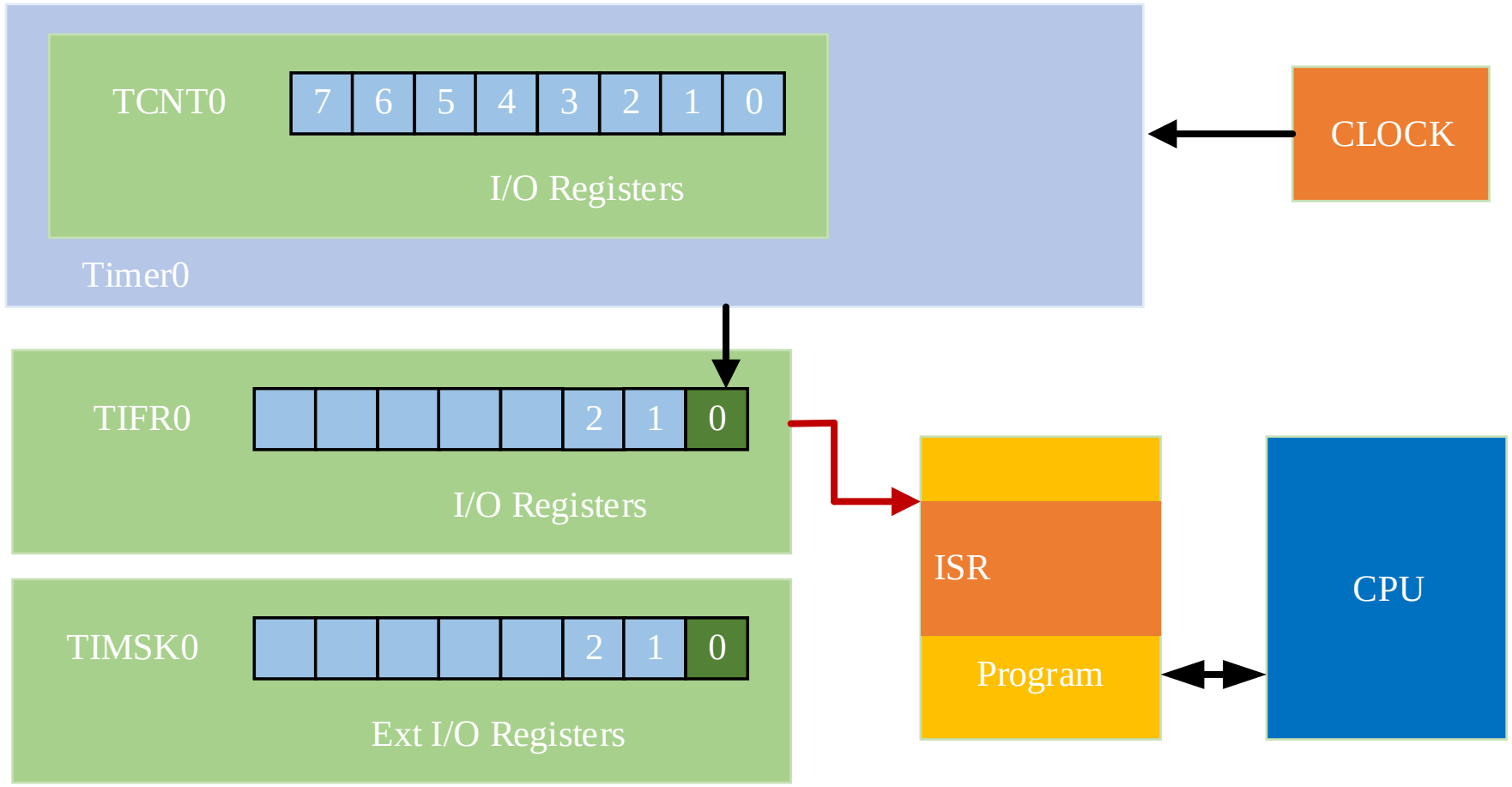




Timer ISR - II

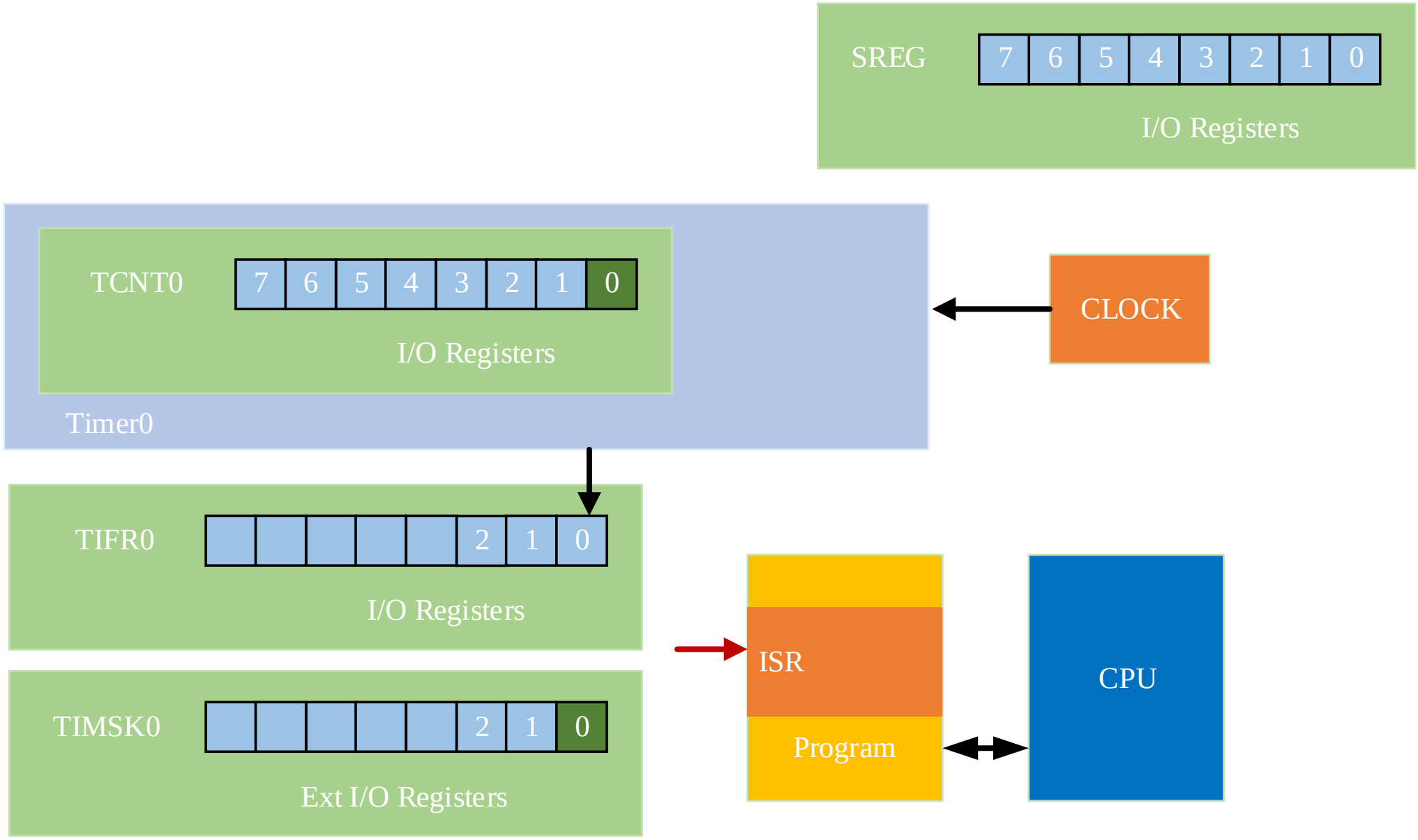
- Overflow back to 0
- Set TOV0 – trigger interrupt
- Disable all other interrupts



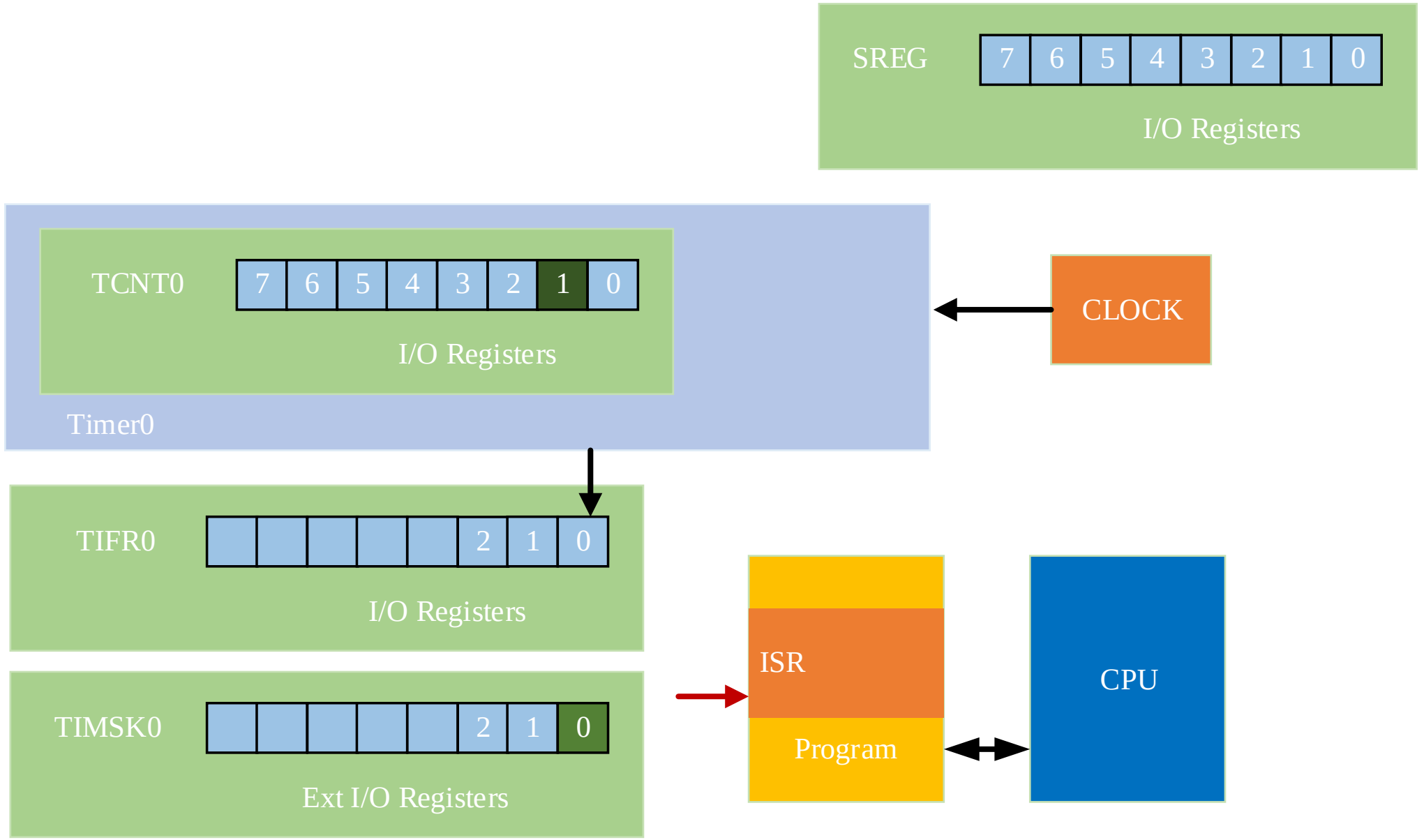


Timer ISR - III

- Run ISR

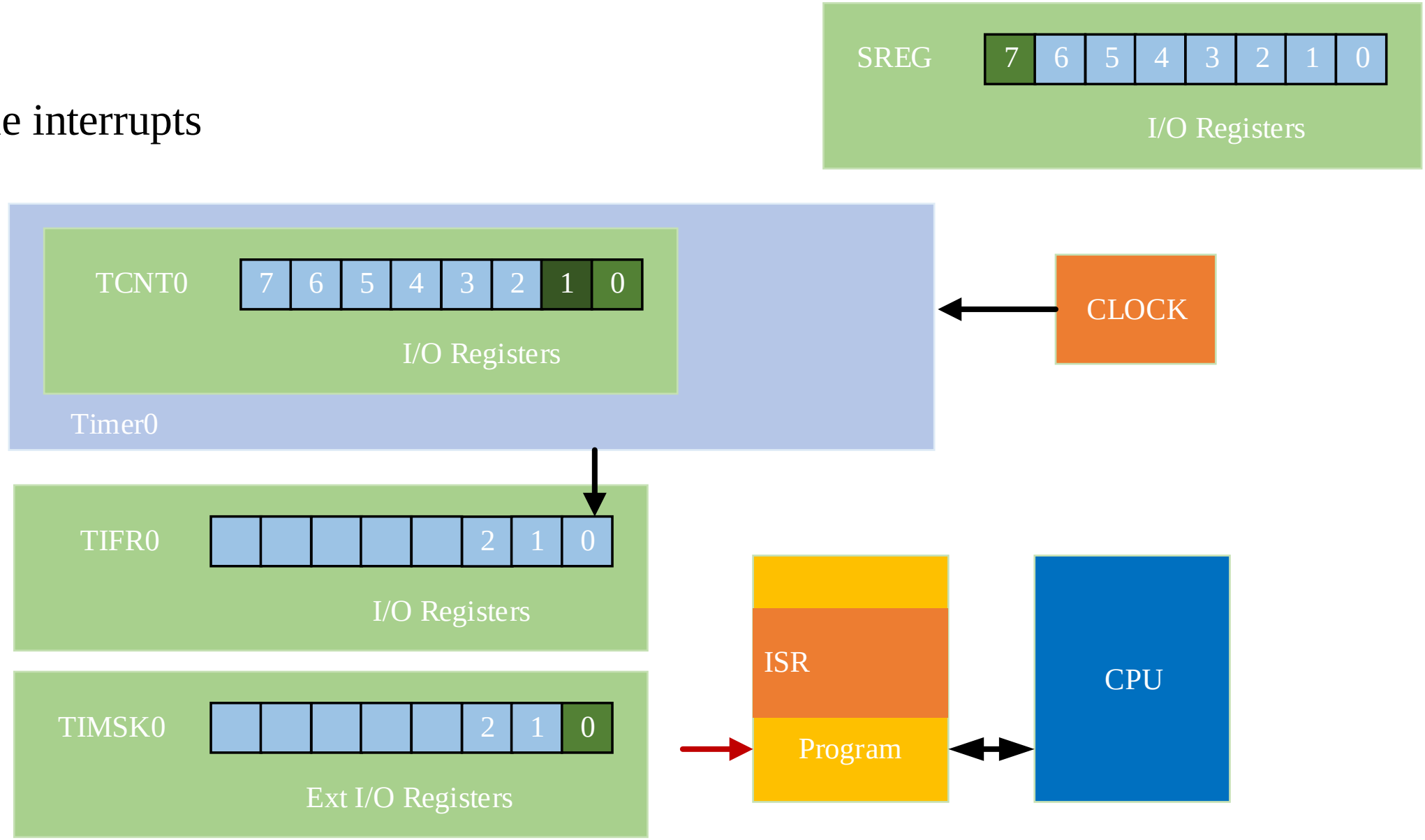


Timer ISR - IV


SREG
7 6 5 4 3 2 1 0
I/O Registers
TCNT0
7 6 5 4 3 2 1 0
I/O Registers
Timer0
CLOCK
TIFR0
2 1 0
I/O Registers
TIMSK0
2 1 0
Ext I/O Registers
ISR
Program
CPU

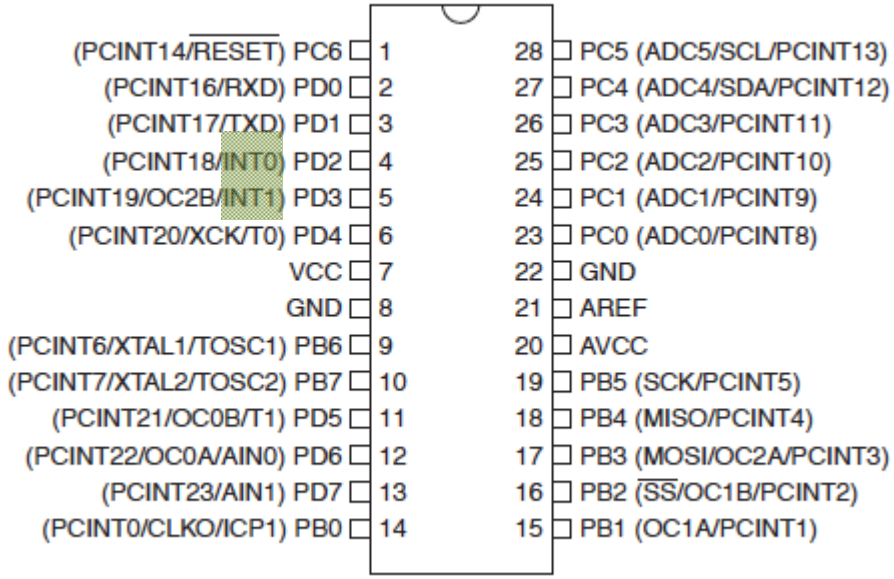
Timer ISR - V

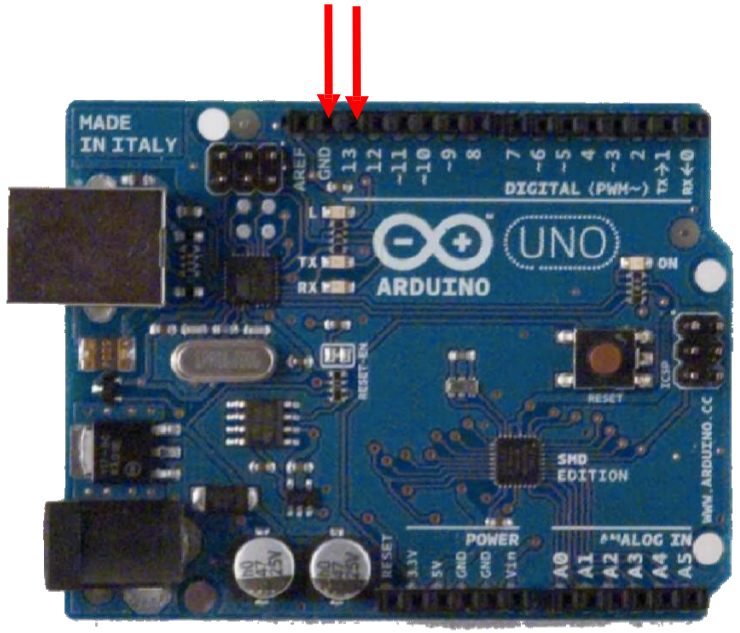
- End ISR
- Re enable interrupts



External Interrupts

- Previous interrupts were internal
- What if we want to react to a button press?
- We could use the INT0 and INT1 pins, dedicated hardware interrupts OR Digital Pins 2,3





Summary

- Polling or delay() brings microcontroller to a halt
- Internal interrupts
  - Reset
  - Timers
- Timer interrupt
  - Write to registers
  - Define ISR
- External interrupt
  - Dedicated pins 2 and 3
  - attachInterrupt()